BLOCKCHAIN LABORATORY

Experiment No. 6

Creation and Deployment of Smart Contract and Transactions using Solidity and Remix IDE

Name: Nikhil Sunchu

Roll No.: 31

Class/section: D20A

Date: _____

1. Aim

Creation and deployment of a smart contract and execution of transactions using Solidity and Remix IDE.

2. Theory

2.1. 1. Smart Contracts and Solidity

A smart contract is a program deployed on a blockchain that executes predefined operations when its functions are called. Solidity is a high-level, statically typed programming language commonly used to develop smart contracts for Ethereum-compatible blockchain platforms. Remix IDE provides a browser-based environment for writing, compiling, deploying, and testing Solidity contracts.

2.2. 2. State Variables and Constructor

State variables store data associated with a deployed contract. In this experiment, `studentName`, `studentRoll`, and `transactionCount` store a student's name, roll number, and the number of update-function calls. The constructor executes once during deployment and initializes the student record using the supplied arguments.

2.3. 3. Functions and Visibility

Functions define the behaviour of a smart contract. The `public` visibility specifier allows functions to be called externally. The `updateName()` and `updateRoll()` functions modify stored state. Public state variables also have automatically generated getter functions, allowing their values to be read through Remix.

2.4. 4. Deployment and Transactions

Compilation checks the source code and generates contract artifacts, including bytecode and an ABI. Deployment creates a contract instance in the selected blockchain environment. A state-changing function call is submitted as a transaction and updates the contract's state. In Remix VM, transactions are processed on a simulated blockchain and do not require real Ether.

2.5. 5. Calls and Transaction Count

Reading a public state variable through its getter is a read operation and does not modify contract state. In this contract, each successful call to `updateName()` or `updateRoll()` in-

crements `transactionCount`. The counter therefore demonstrates the number of update-function executions in the experiment; it does not count the deployment or getter calls.

3. Tools and Environment Used

- Remix IDE (browser-based development environment)
- Solidity compiler, version 0.8.34 as shown in the compiler panel
- Remix VM with the Osaka VM setting
- Solidity source file: `StudentRecord.sol`
- Remix Solidity Compiler and Deploy & Run Transactions panels

4. Tasks Performed

4.1. Task 1 – Write the Solidity Program

- Created a Solidity source file named `StudentRecord.sol` in Remix IDE.
- Defined the `StudentRecord` contract with student name, roll number, and transaction count variables.
- Used a constructor to initialize the student record during deployment.
- Implemented `updateName()` and `updateRoll()` to modify the stored values.
- Incremented `transactionCount` after each update.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract StudentRecord {
    string public studentName;
    uint256 public studentRoll;
    uint256 public transactionCount;

    constructor(string memory _name, uint256 _roll) {
        studentName = _name;
        studentRoll = _roll;
        transactionCount = 0;
    }

    function updateName(string memory _name) public {
        studentName = _name;
        transactionCount++;
    }

    function updateRoll(uint256 _roll) public {
        studentRoll = _roll;
        transactionCount++;
    }
}
```

4.2. Task 2 – Compile the Contract

- Opened the Solidity Compiler panel in Remix IDE.
- Selected a compatible Solidity 0.8.x compiler.
- Compiled `StudentRecord.sol` and confirmed successful compilation.
- The source code and compiler panel are shown in Figure 1.

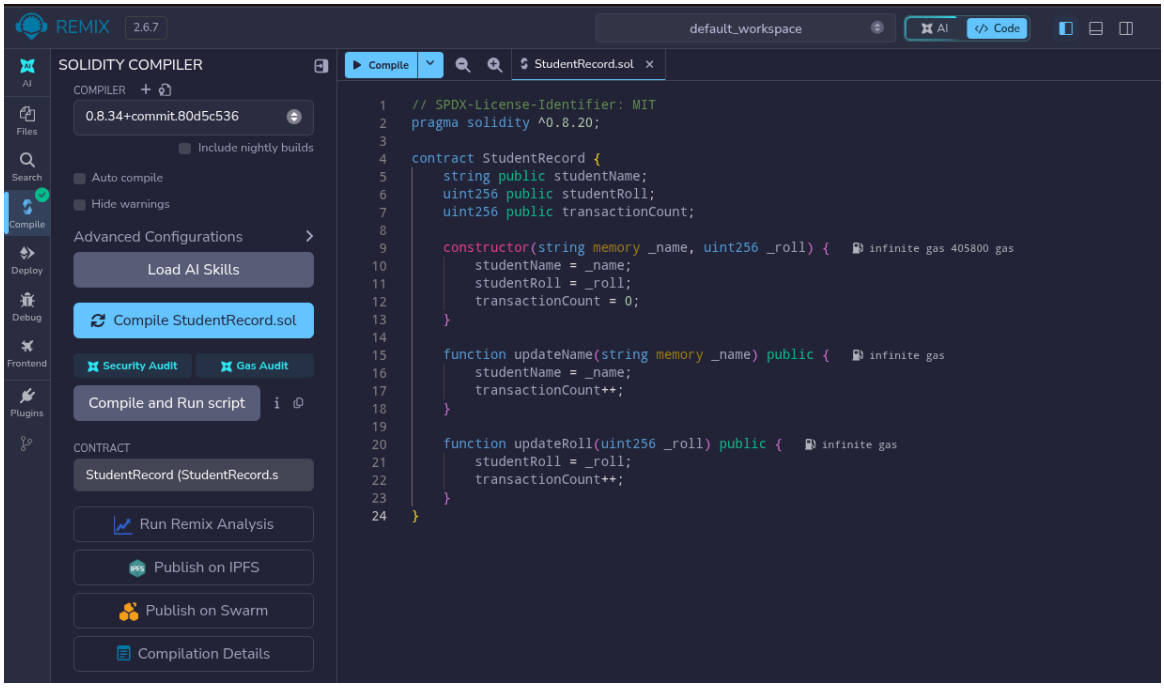


Figure 1: StudentRecord source code and successful compilation in Remix IDE

4.3. Task 3 – Deploy the Contract

- Opened the Deploy & Run Transactions panel.
- Selected Remix VM as the execution environment.
- Supplied `Nikhil Sunchu` and `31` as constructor arguments.
- Deployed the `StudentRecord` contract and confirmed that it appeared under Deployed Contracts.
- The deployment and decoded constructor arguments are shown in Figure 2.

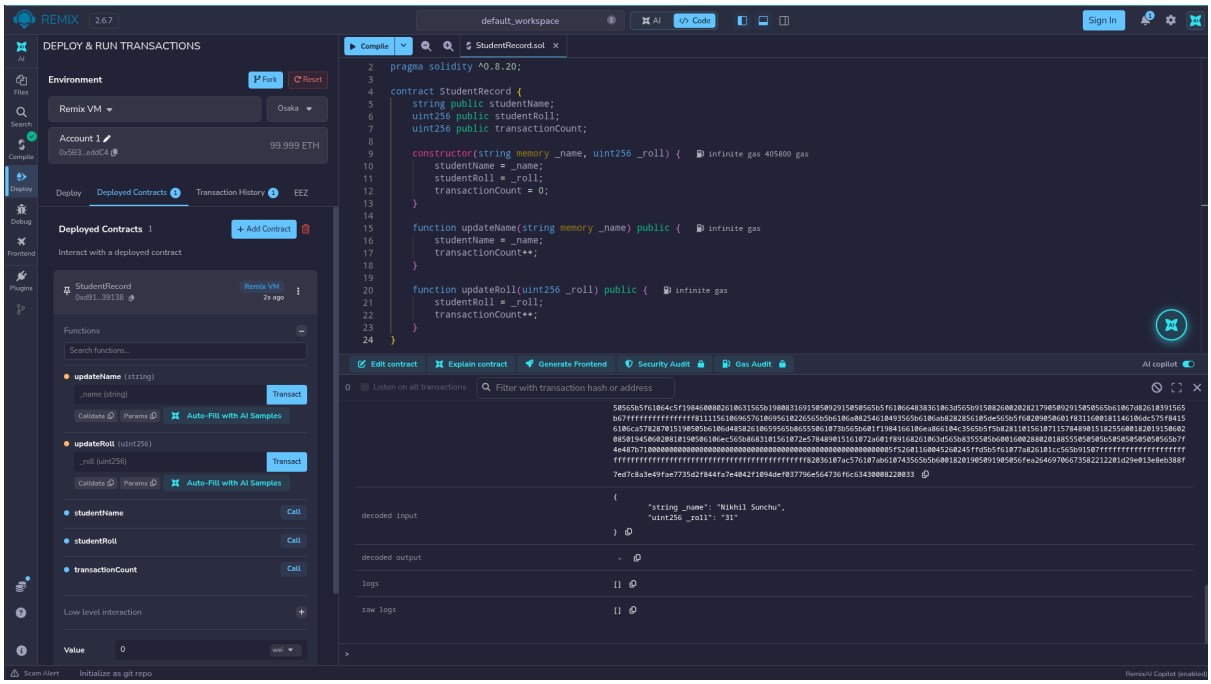


Figure 2: Deployment of StudentRecord in Remix VM

4.4. Task 4 – Execute State-Changing Transactions

- Entered `Nikhil Bikshapati Sunchu` as the argument to `updateName()` and executed the transaction.
- Entered `32` as the argument to `updateRoll()` and executed the transaction.
- Used the Transact buttons to submit both state-changing function calls.
- Remix VM processed the calls as transactions on the simulated blockchain.

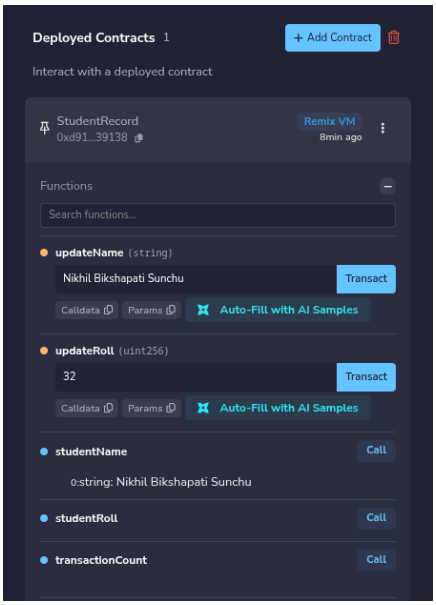


Figure 3: Execution of updateName() and updateRoll() transactions

4.5. Task 5 – Verify the Updated Values

- Called the `studentName()` getter and observed `Nikhil Bikshapati Sunchu`.
- Called the `studentRoll()` getter and observed `32`.
- Called the `transactionCount()` getter and observed `2`.
- Confirmed that both update functions modified the stored values and incremented the counter.
- The returned values are shown in Figure 4.

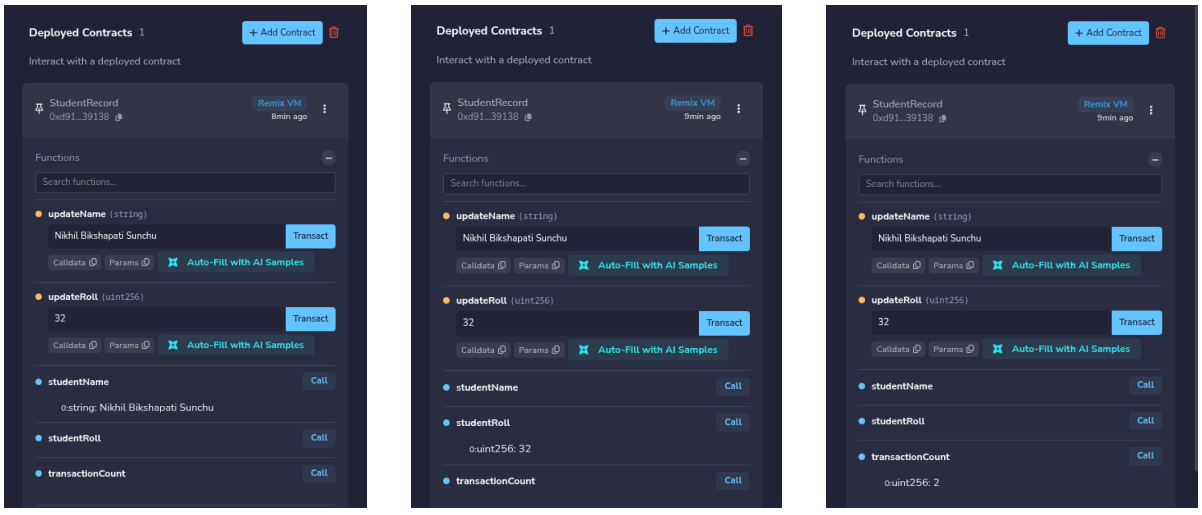


Figure 4: Verification of the updated student name, roll number, and transaction count

5. Result

The `StudentRecord` smart contract was successfully written in Solidity, compiled, and deployed using Remix IDE and Remix VM. The stored student name was updated to `Nikhil Bikshapati Sunchu`, and the roll number was updated to `32`. The `transactionCount` getter returned `2`, confirming that both state-changing update functions were executed successfully.

6. Conclusion

The experiment provided practical experience in creating, compiling, and deploying a Solidity smart contract using Remix IDE. Interacting with the deployed `StudentRecord` contract demonstrated constructor-based initialization, public state variables, state-changing functions, and getter calls. The experiment also illustrated how transactions can update contract state in a simulated blockchain environment without using real cryptocurrency.